INTRODUCTION TO LINUX SHELL AND SHELL SCRIPTING

If we are using any major operating system, we are indirectly interacting with the **shell**. While running Ubuntu, Linux Mint, or any other Linux distribution, we are interacting with the shell by using the terminal. In this article we will discuss Linux shells and shell scripting so before understanding shell scripting we have to get familiar with the following terminologies:

- Kernel
- Shell
- Terminal

What is Kernel?

The kernel is a computer program that is the core of a computer's operating system, with complete control over everything in the system. It manages the following resources of the Linux system –

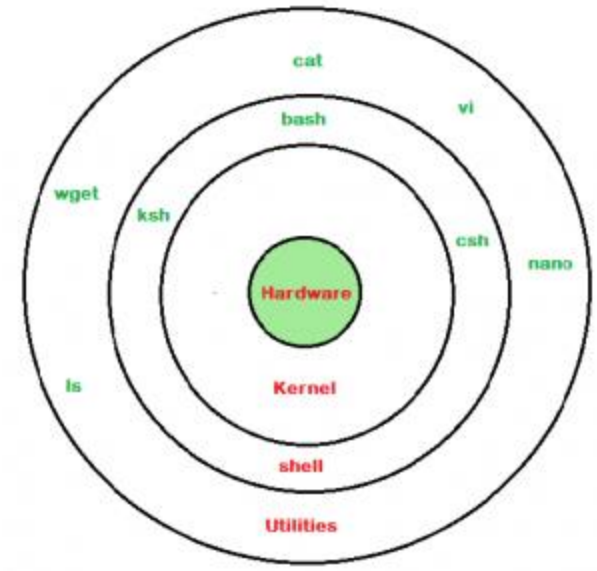
- File management
- Process management
- I/O management
- Memory management
- Device management etc.

It is often mistaken that Linus Torvalds has developed Linux OS, but actually, he is only responsible for the development of the Linux kernel.

Complete Linux system = Kernel + GNU system utilities and libraries + other management scripts + installation scripts.

What is Shell?

A shell is a special user program that provides an interface for the user to use operating system services. Shell accepts human-readable commands from users and converts them into something which the kernel can understand. It is a command language interpreter that executes commands read from input devices such as keyboards or from files. The shell gets started when the user logs in or starts the terminal.



Linux Shell

Shell is broadly classified into two categories –

- Command Line Shell
- Graphical shell

Command Line Shell

Shell can be accessed by users using a command line interface. A special program called Terminal in Linux/macOS, or Command Prompt in Windows OS is provided to type in the human-readable commands such as “cat”, “ls” etc. and then it is being executed. The result is then displayed on the terminal to the user. A terminal in Ubuntu 16.4 system looks like this –

linux command line

In the above screenshot “**ls**” command with “**-l**” option is executed. It will list all the files in the current working directory in a long listing format.
Working with a command line shell is a bit difficult for beginners because it’s hard to memorize so many commands. It is very powerful; it allows users to store commands in a file and execute them together. This way any repetitive task can be easily automated. These files are usually called batch files in Windows and **Shell** Scripts in Linux/macOS systems.

Graphical Shells

Graphical shells provide means for manipulating programs based on the graphical user interface (GUI), by allowing for operations such as opening, closing, moving, and resizing windows, as well as switching focus between windows. Window OS or Ubuntu OS can be considered as a

GUI Shell

There are several shells are available for Linux systems like –

- BASH (Bourne Again SHell) – It is the most widely used shell in Linux systems. It is used as default login shell in Linux systems and in macOS. It can also be installed on Windows OS.
- CSH (C SHell) – The C shell's syntax and its usage are very similar to the C programming language.
- KSH (Korn SHell) – The Korn Shell was also the base for the POSIX Shell standard specifications etc.

Each shell does the same job but understands different commands and provides different built-in functions.

What is a terminal?

A program which is responsible for providing an interface to a user so that he/she can access the shell. It basically allows users to enter commands and see the output of those commands in a text-based interface. Large scripts that are written to automate and perform complex tasks are executed in the terminal.

To access the terminal, simply search in search box "terminal" and double-click it.

The Linux command is a utility of the Linux operating system. All basic and advanced tasks can be done by executing commands. The commands are executed on the **Linux terminal**. The terminal is a command-line interface to interact with the system, which is similar to the command prompt in the Windows OS. *Commands in Linux are **case-sensitive**.*

Linux provides a powerful command-line interface compared to other operating systems such as Windows and MacOS. We can do basic work and advanced work through its terminal. We can do some basic tasks such as creating a file, deleting a file, moving a file, and more. In addition, we can also perform advanced tasks such as administrative tasks (including package installation, user management), networking tasks (ssh connection), security tasks, and many more.

Linux terminal is a user-friendly terminal as it provides various support options. To open the Linux terminal, press "**CTRL + ALT + T**" keys together, and execute a command by pressing the '**ENTER**' key.

These commands are very useful for a beginner and professional both. We have divided these commands into following sections so that you can easily identify their usage:

- Linux Directory Commands
- Linux File Commands
- Linux File Content Commands
- Linux User Commands
- Linux Filter Commands
- Linux Utility Commands
- Linux Networking Command

Linux Directory Commands

1. pwd Command

The pwd command is used to display the location of the current working directory.

Syntax:

```
pwd
```

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ pwd
/home/javatpoint
```

mkdir Command

The mkdir command is used to create a new directory under any directory.

Syntax:

mkdir <directory name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ mkdir new_directory
javatpoint@javatpoint-Inspiron-3542:~$
```

rmdir Command

The rmdir command is used to delete a directory.

Syntax:

rmdir <directory name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ rmdir new_directory
javatpoint@javatpoint-Inspiron-3542:~$
```

ls Command

The ls command is used to display a list of content of a directory.

Syntax:

ls

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ ls
a               Desktop            examples.desktop  Music           sample
Akash           Directory          hello.c           pico            snap
a.out           Documents          hello.i           Pictures        Templates
composer.phar   Downloads          hello.o           project         Test.txt
Demo.sh         eclipse            hello.s           Public          Videos
Demo.txt        eclipse-installer  index.html        Python
Demo.txt~       eclipse-workspace  mail              Python-3.8.0
```

cd Command

The cd command is used to change the current directory.

Syntax:

cd <directory name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ cd Desktop
javatpoint@javatpoint-Inspiron-3542:~/Desktop$
```

Linux File commands
touch Command

The touch command is used to create empty files. We can create multiple empty files by executing it once.

Syntax:

1. touch <file name>
2. touch <file1> <file2> ....

Output:

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ touch Demo.txt
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ touch Demo1.txt Demo2.txt
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ ls
Demo1.txt  Demo2.txt  Demo.txt
```

cat Command

The cat command is a multi-purpose utility in the Linux system. It can be used to create a file, display content of the file, copy the content of one file to another file, and more.

Syntax:

cat [OPTION]... [FILE]..

To create a file, execute it as follows:

1. cat > <file name>
2. // Enter file content

Press "**CTRL+ D**" keys to save the file. To display the content of the file, execute it as follows:

cat <file name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ cat > Demo.txt
This is a text file.
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ cat Demo.txt
This is a text file.
```

rm Command

The rm command is used to remove a file.

Syntax:

rm <file name>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ rm Demo.txt
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ rm Demo1.txt Demo2.txt
```

cp Command

The cp command is used to copy a file or directory.

Syntax:

To copy in the same directory:

cp <existing file name> <new file name>

To copy in a different directory:

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ cp demo.txt demo1.txt
javatpoint@javatpoint-Inspiron-3542:~$ cp demo.txt Documents
```

mv Command

The mv command is used to move a file or a directory form one location to another location.

Syntax:

mv <file name> <directory path>

Output:

```
javatpoint@javatpoint-Inspiron-3542:~$ mv demo.txt Directory
```

Input Output Redirection in Linux

In Linux, whenever an individual runs a command, it can take input, give output, or do both. Redirection helps us redirect these input and output functionalities to the files or folders we want, and we can use special commands or characters to do so.

For example, if we run the “date” command, it gives us output on the screen. However, if we want to save the result in a file instead, we can use output redirection. This way, we can store the output of the date command in a file and refer to it later. These redirections can come in handy when we work with multiple and large outputs or inputs since we can use file data directly as input and store results in files.

All this can be done easily on the terminal using some simple commands. In this article, we will talk about different types of Redirections in linux and how to use Redirections in linux.

Types of Redirection

1. Overwrite Redirection:

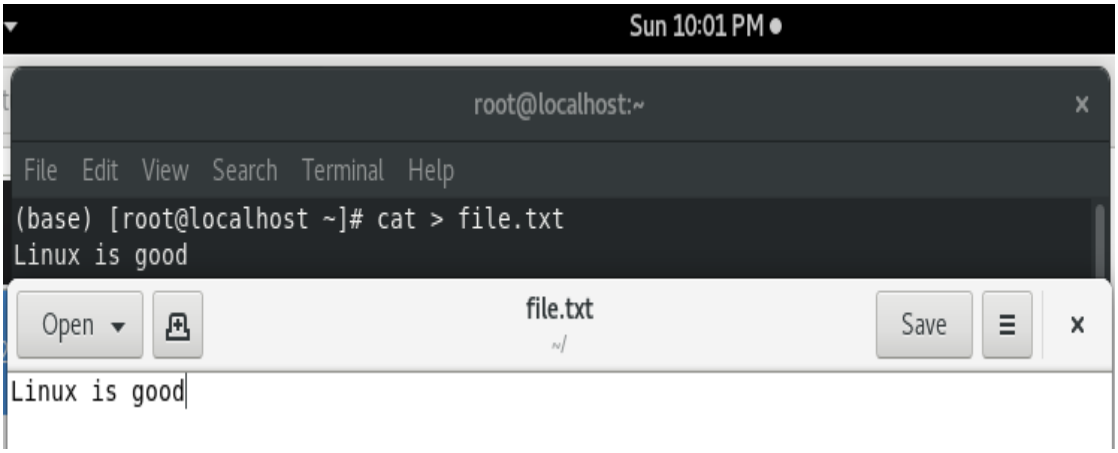
Overwrite redirection is useful when you want to store/save the output of a command to a file and replace all the existing content of that file. for example, if you run a command that gives a report, and you want to save the report to the existing file of the previous report you can use overwrite redirection to do this.

- “>” standard output
- “<” standard input

Implementation:

So whatever you will write after running this command, will be redirected and copied to the “file.txt”. This is standard **output redirection.**

```
cat > file.txt
```



Now, this is standard **input redirection**, cat command will take the input from "file.txt" and print it to the terminal screen. This line of code also shows the real working and meaning of the cat command that is copy and paste. Many people have a misconception that the cat is used to create a file, but it is not true, the main work of the cat is to copy the input and give the output to the screen.

```
cat < file.txt
```

Just type cat on the terminal and hit enter. It will ask for the input lines, you could write your name and hit enter. You will see your input will be reprinted.

```
(base) [root@localhost ~]# cat
Hello this is Linux
Hello this is Linux
```

This is used when we want to append some lines to the existing content of the file. If you use only a single angular bracket all the content of the file will be lost.

```
cat >> file.txt
```

Append Redirection:

With the help of this Redirection, you can append the output to the file without compromising the existing data of the file.

- ">>" standard output
- "<<" standard input

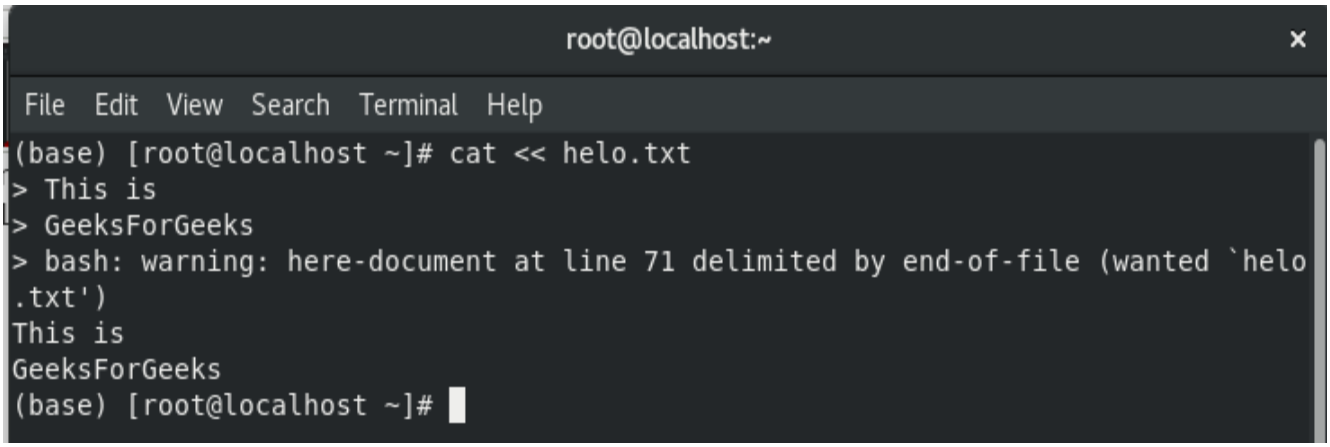
The general form for a here document is −

```
Syntax:
command << delimiter
document
delimiter

(base) [root@localhost ~]# cat << helo.txt
> Hello This is
> GeeksForGeeks
helo.txt
Hello This is
GeeksForGeeks
(base) [root@localhost ~]#

Note: Here, helo.txt is a delimiter.
```

The delimiter marks the ending point of the document. Without it, the shell continues to read the input forever. The delimiter must be a single word that does not contain spaces or tabs.



Merge Redirection:

This allows you to redirect the output of a command or a program to a specific file descriptor instead of standard output. the syntax for using this is ">&" operator followed by the file descriptor number.

- "p >& q" Merges output from stream p with stream q
- "p <& q" Merges input from stream p with stream q

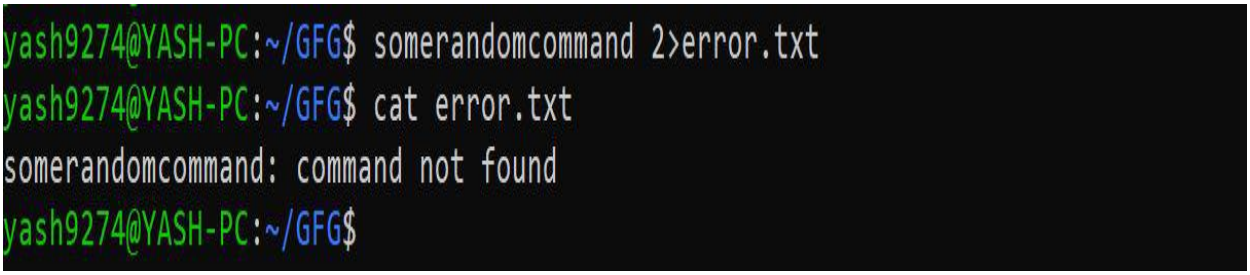
Error Redirection: Error redirection is transferring the errors generated by some false commands to a file rather than STDOUT.

Whenever a program is executed at the terminal, 3 files are generated: standard input(0), standard output(1), standard error(2). These files are always created whenever a program is run. By default, an error stream is displayed on the screen.

Examples:

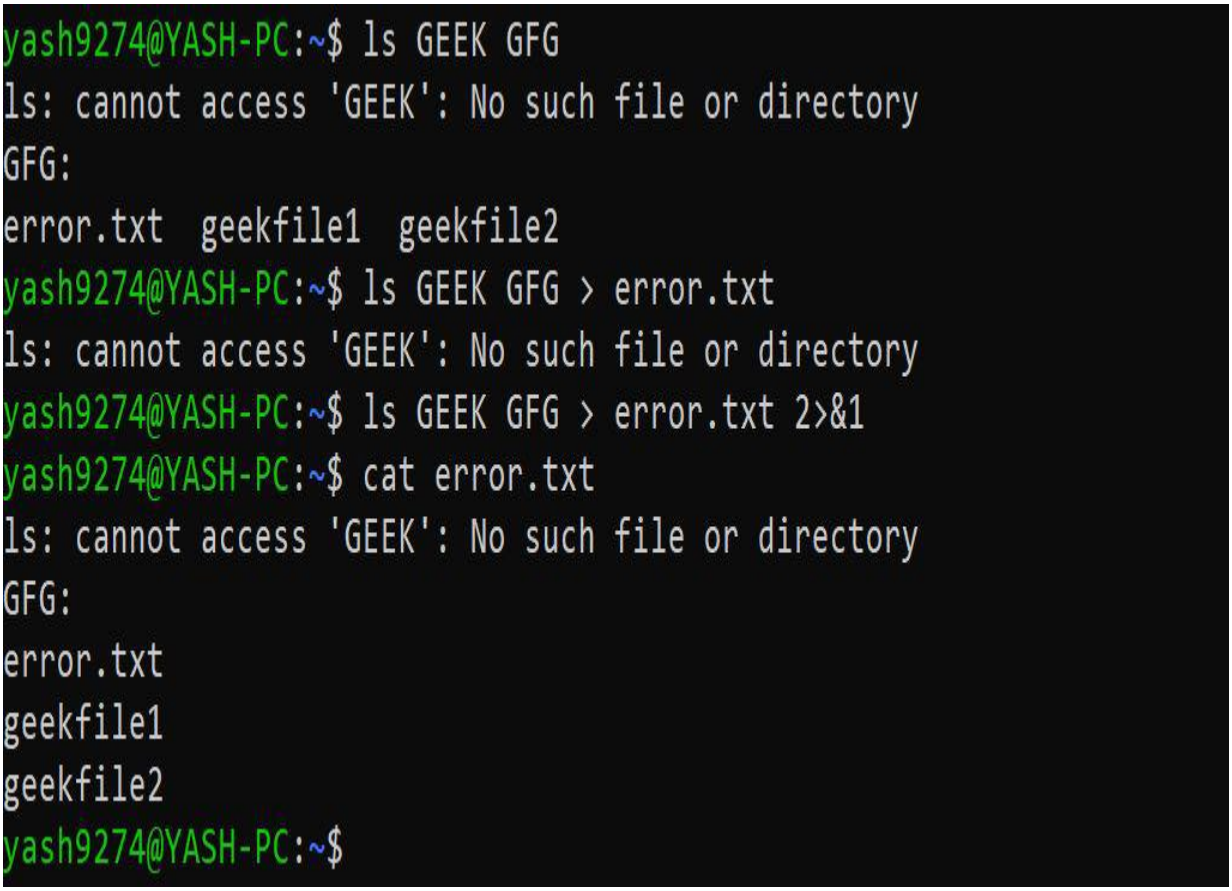
1. In the below-mentioned example, the file descriptor used above is **2(STDERR)**. Using "**2>**" re-directs the error output to a file named "error.txt" and nothing is displayed on **STDOUT.**

```
$ somerandomcommand 2>error.txt
```



2. Here, **2>&1** means that **STDERR** redirects to the target of **STDOUT.** More formally, the error message generated by "**2**" gets merged with the current output "**1**".

```
$ ls GEEK GFG > error.txt 2>&1
```



In the above example, the directory **GEEK** is not present. The error output is merged with the standard output which in turn is being re-directed to "**error.txt**".

Piping in Unix or Linux

A pipe is a form of redirection (transfer of standard output to some other destination) that is used in Linux and other Unix-like operating systems to send the output of one command/program/process to another command/program/process for further processing. The Unix/Linux systems allow the stdout of a command to be connected to the stdin of another command. You can make it do so by using the pipe character '|'.

The pipe is used to combine two or more commands, and in this, the output of one command acts as input to another command, and this command's output may act as input to the next command, and so on. It can also be visualized as a temporary connection between two or more commands/ programs/ processes. The command line programs that do the further processing are referred to as filters.

This direct connection between commands/ programs/ processes allows them to operate simultaneously and permits data to be transferred between them continuously rather than having to pass it through temporary text files or through the display screen. Pipes are unidirectional i.e., **data flows from left to right through the pipeline.**

Syntax:

```
command_1 | command_2 | command_3 | .... | command_N
```

List all files and directories and give them as input to `grep` command using piping in Linux

```
ls | grep file.txt
```

ls | grep file.txt

In this first we are using `ls` to list all file and directories in the current directory, then passing its output to `grep` command and searching for file name `file.txt`. The output of the ls command is sent to the input of the grep command, and the result is a list of files that match the search term.

List all files and directories and give them as input to `more` commands using piping in Linux.

```
$ ls -l | more
```

$ ls -l | more

The more command takes the output of $ ls -l as its input. The net effect of this command is that the output of ls -l is displayed one screen at a time. The pipe acts as a container which takes the output of ls -l and gives it to more as input. This command does not use a disk to connect standard output of ls -l to the standard input of more because pipe is implemented in the main memory.

Filters in Linux

Filters are programs that take plain text(either stored in a file or produced by another program) as standard input, transforms it into a meaningful format, and then returns it as standard output. Linux has a number of filters. Some of the most commonly used filters are explained below:

cat : Displays the text of the file line by line.

Syntax:

```
cat [path]
```

head : Displays the first n lines of the specified text files. If the number of lines is not specified then by default prints first 10 lines.

Syntax:

```
head [-number_of_lines_to_print] [path]
```

tail : It works the same way as head, just in reverse order. The only difference in tail is, it returns the lines from bottom to up.

Syntax:

```
tail [-number_of_lines_to_print] [path]
```

sort : Sorts the lines alphabetically by default but there are many options available to modify the sorting mechanism. Be sure to check out the main page to see everything it can do.

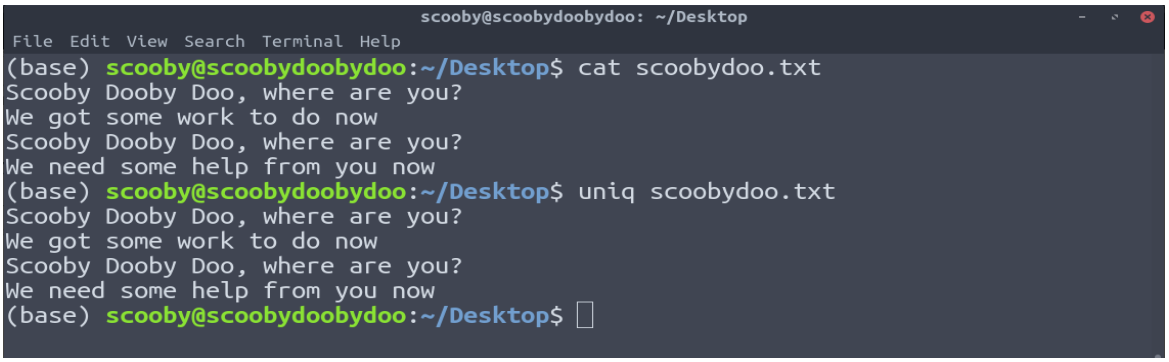
Syntax:

```
sort [-options] [path]
```

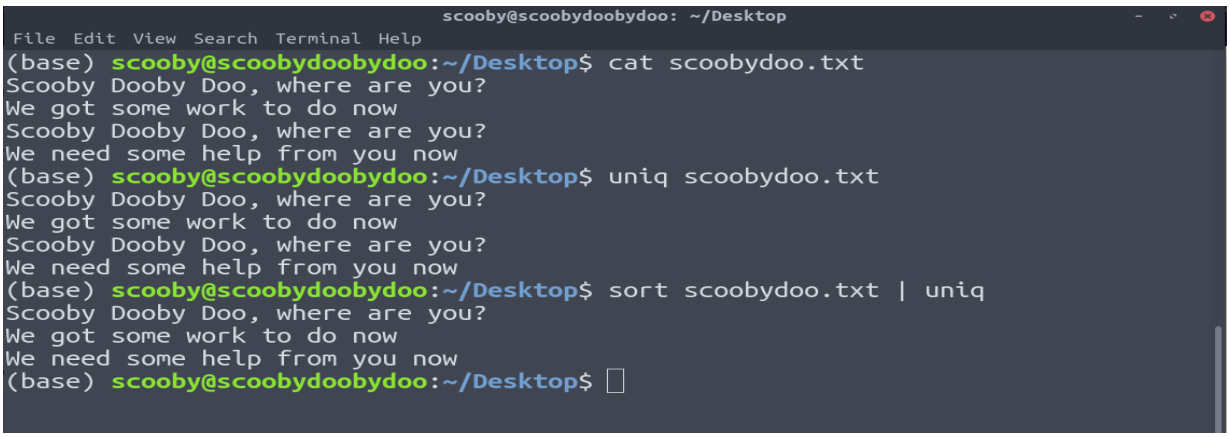
uniq : Removes duplicate lines. uniq has a limitation that it can only remove continuous duplicate lines(although this can be fixed by the use of piping). Assuming we have the following data.

Syntax:

```
uniq [options] [path]
```

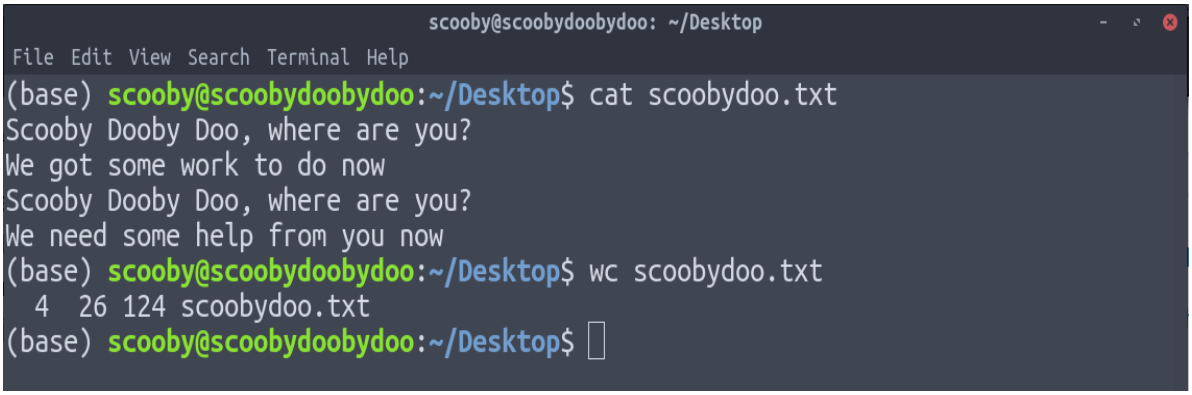


You can see that applying uniq doesn't remove any duplicate lines, because uniq only removes duplicate lines which are together.



When applying uniq to sorted data, it removes the duplicate lines because, after sorting data, duplicate lines come together.

wc : wc command gives the number of lines, words and characters in the data.
Syntax:
`wc [-options] [path]`

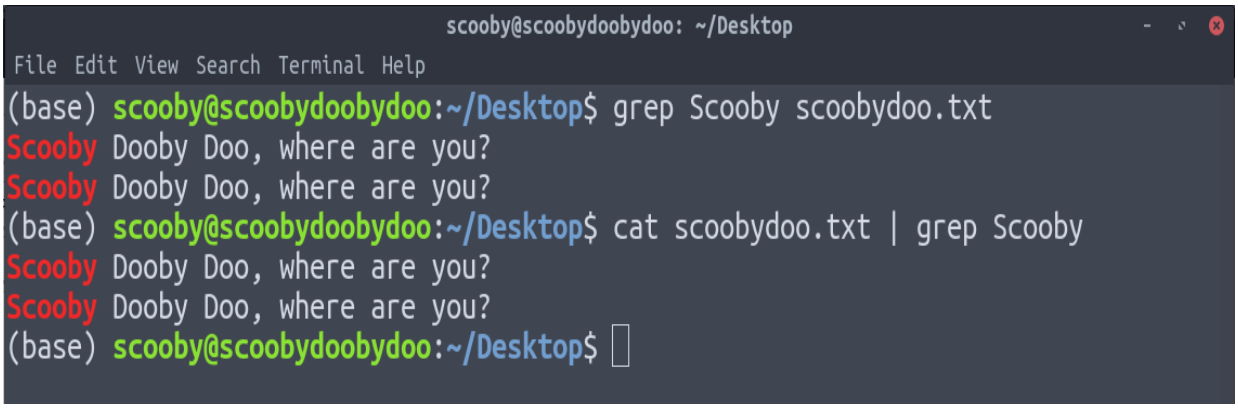


In above image the wc gives 4 outputs as:

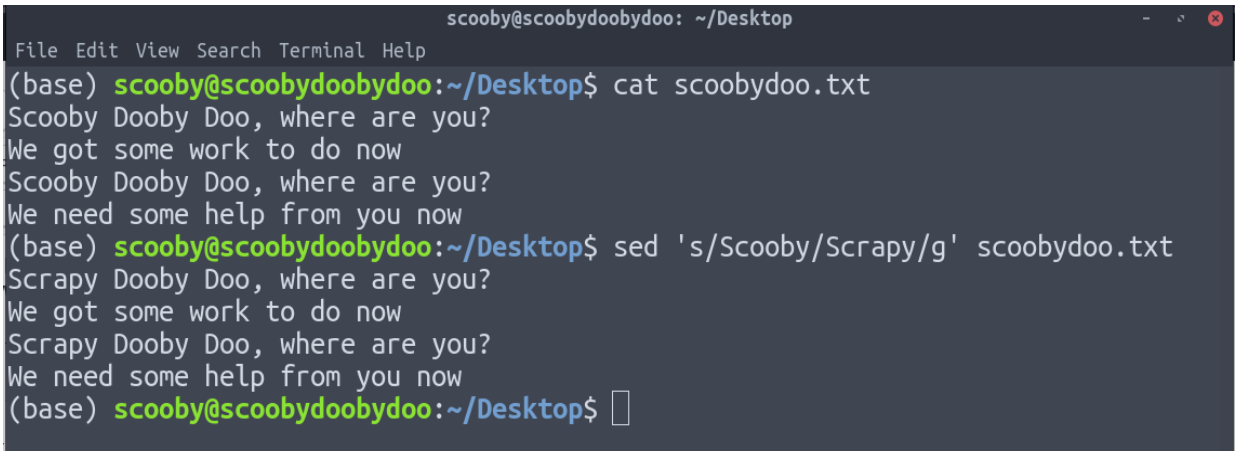
- number of lines
- number of words
- number of characters
- path

grep : grep is used to search a particular information from a text file.
Syntax:
`grep [options] pattern [path]`

Below are the two ways in which we can implement grep.



tac : tac is just the reverse of cat and it works the same way, i.e., instead of printing from lines 1 through n, it prints lines n through 1. It is just reverse of cat command.
Syntax:
`tac [path]`

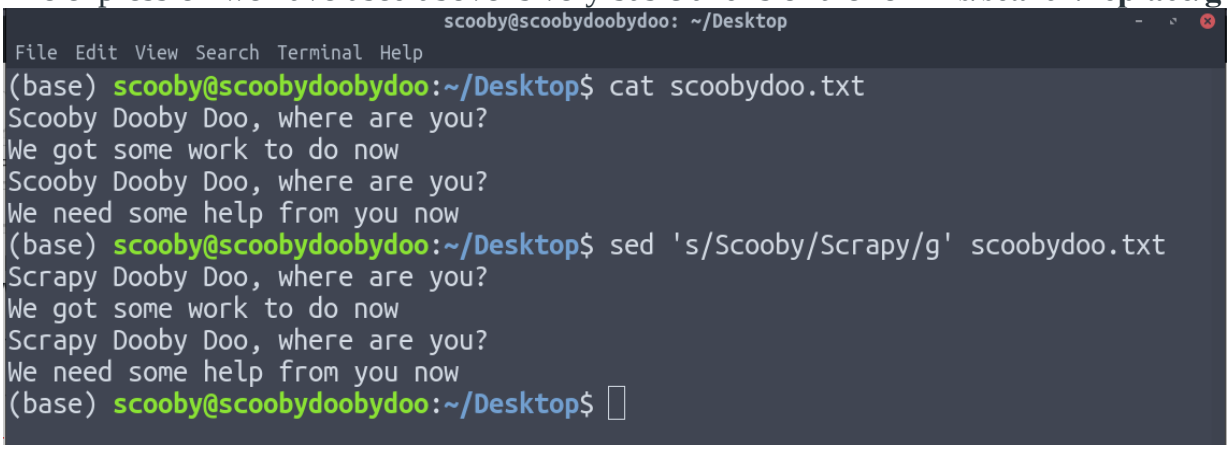


sed : sed stands for stream editor. It allows us to apply search and replace operation on our data effectively. *sed* is quite an advanced filter and all its options can be seen on its man page.

Syntax:

```
sed  [path]
```

The expression we have used above is very basic and is of the form **'s/search/replace/g'**

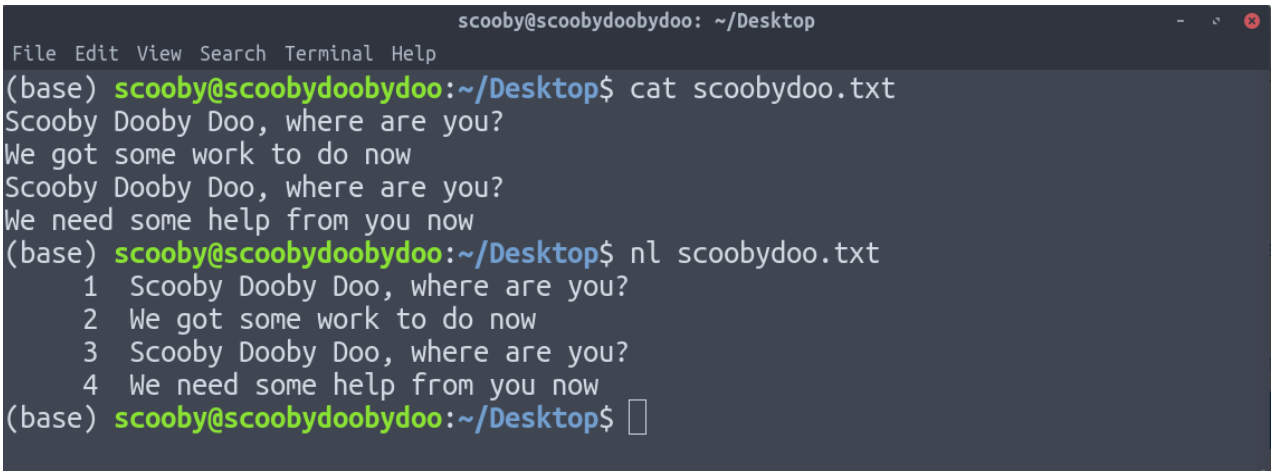


In the above image, we can clearly see that Scooby is replaced by Scrapy.

nl : nl is used to number the lines of our text data.

Syntax:

```
nl [-options] [path]
```



Special Characters

#

Comments. Lines beginning with a # (with the exception of #!) are comments and will *not* be executed.

```
# This line is a comment.
```

Comments may also occur following the end of a command.

```
echo "A comment will follow." # Comment here.
#                             ^ Note whitespace before #
```

Comments may also follow whitespace at the beginning of a line.

```
     # A tab precedes this comment.
```

Comments may even be embedded within a pipe.

```
initial=( `cat "$startfile" | sed -e '/#/d' | tr -d '\n' |\
# Delete lines containing '#' comment character.
           sed -e 's/\./\. /g' -e 's/_/_ /g'` )
# Excerpted from life.sh script
```

Command separator [semicolon].

Permits putting two or more commands on the same line.

```
echo hello; echo there
```

```
if [ -x "$filename" ]; then    #  Note the space after the semicolon.
#+                    ^^
  echo "File $filename exists."; cp $filename $filename.bak
else   #                       ^^
  echo "File $filename not found."; touch $filename
fi; echo "File test complete."
```

;;

Terminator in a case option [double semicolon].

```
case "$variable" in
  abc)  echo "\$variable = abc" ;;
  xyz)  echo "\$variable = xyz" ;;
esac
```

.

"dot" command [period].

."**dot", as a component of a filename.** When working with filenames, a leading dot is the prefix of a "hidden" file, a file that an ls will not normally show.

Filters in Linux

Filters are programs that take plain text(either stored in a file or produced by another program) as standard input, transforms it into a meaningful format, and then returns it as standard output. Linux has a number of filters. Some of the most commonly used filters are explained below:

cat : Displays the text of the file line by line.

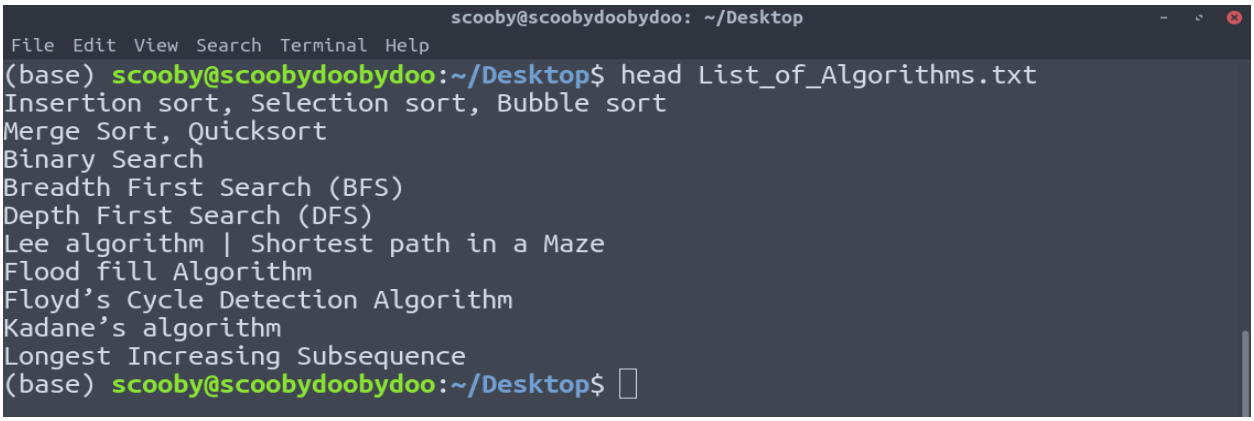
Syntax:

```
cat [path]
```

head : Displays the first n lines of the specified text files. If the number of lines is not specified then by default prints first 10 lines.

Syntax:

```
head [-number_of_lines_to_print] [path]
```





tail : It works the same way as head, just in reverse order. The only difference in tail is, it returns the lines from bottom to up.

Syntax:

```
tail [-number_of_lines_to_print] [path]
```

sort : Sorts the lines alphabetically by default but there are many options available to modify the sorting mechanism. Be sure to check out the main page to see everything it can do.

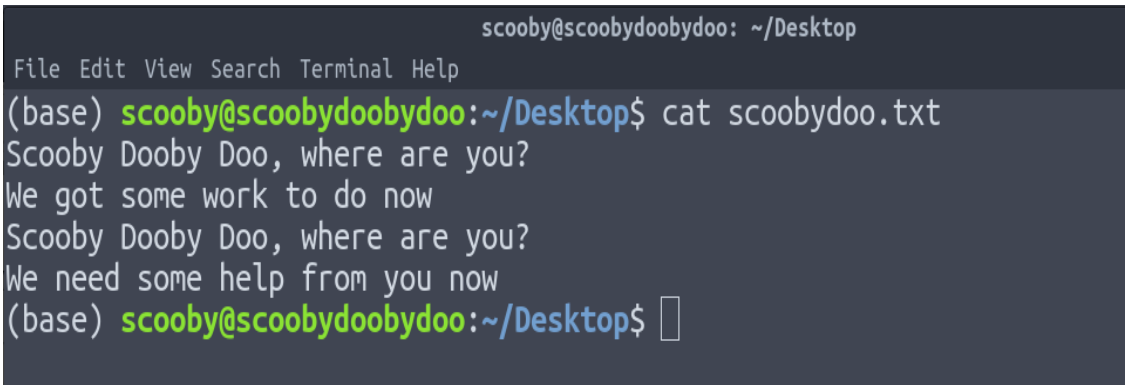
Syntax:

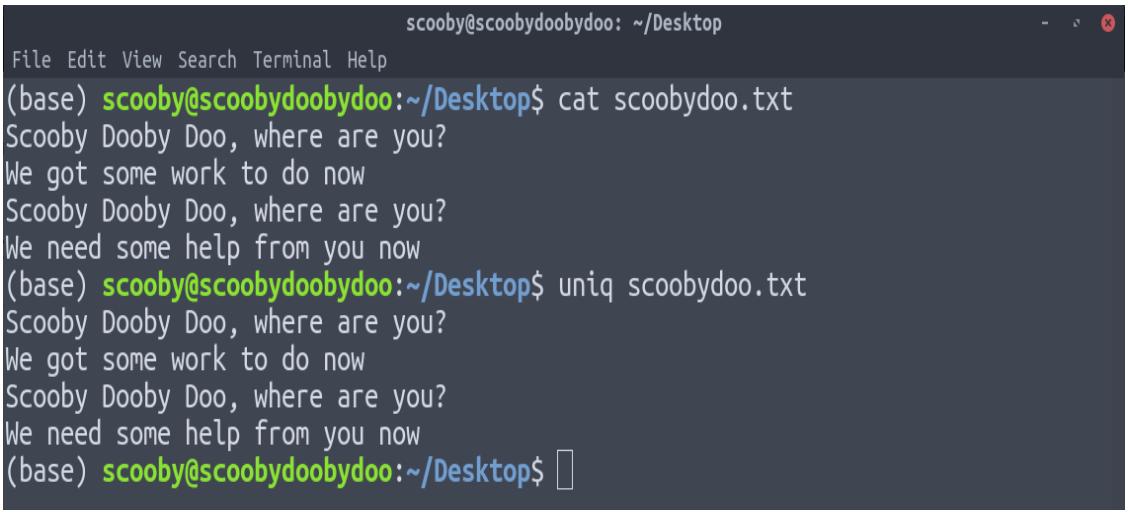
```
sort [-options] [path]
```

uniq : Removes duplicate lines. uniq has a limitation that it can only remove continuous duplicate lines(although this can be fixed by the use of piping). Assuming we have the following data.

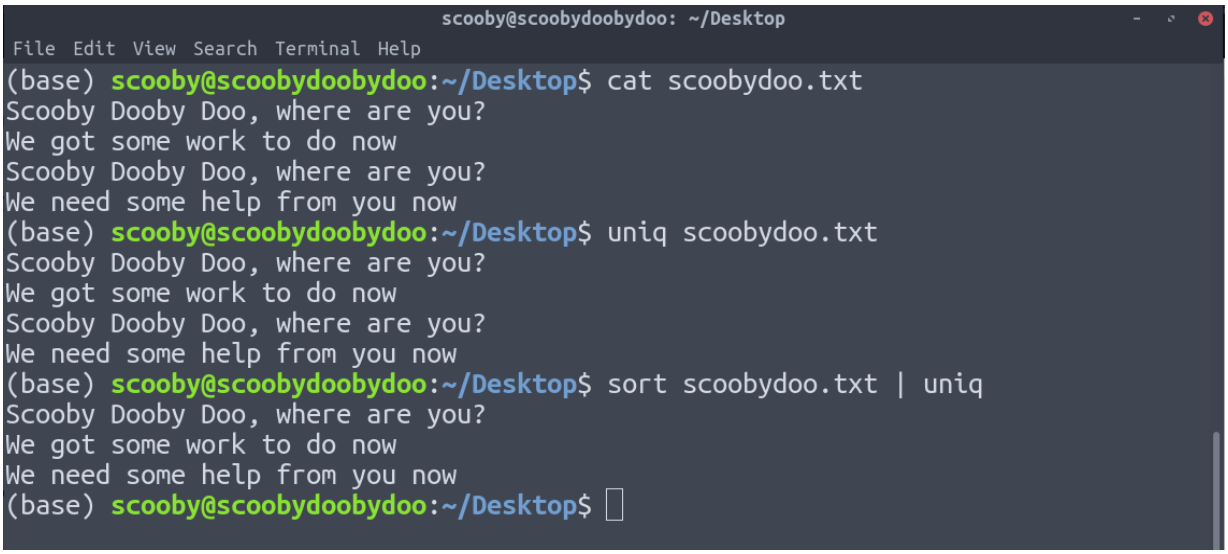
Syntax:

```
uniq [options] [path]
```





You can see that applying uniq doesn't remove any duplicate lines, because uniq only removes duplicate lines which are together.



When applying uniq to sorted data, it removes the duplicate lines because, after sorting data, duplicate lines come together.

wc : wc command gives the number of lines, words and characters in the data.

Syntax:

```
wc [-options] [path]
```

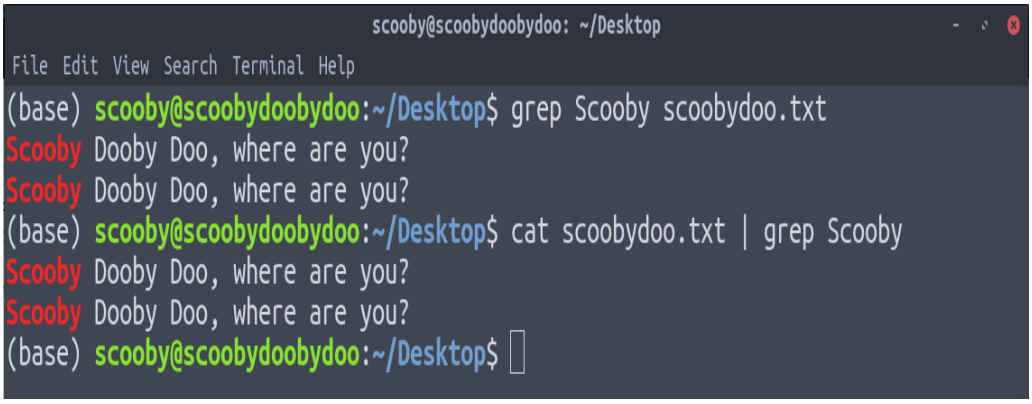
In above image the wc gives 4 outputs as:

- number of lines
- number of words
- number of characters
- path

grep : grep is used to search a particular information from a text file.

Syntax:

```
grep [options] pattern [path]
```



tac : tac is just the reverse of cat and it works the same way, i.e., instead of printing from lines 1 through n, it prints lines n through 1. It is just reverse of cat command.

Syntax:

```
tac [path]
```

sed : sed stands for stream editor. It allows us to apply search and replace operation on our data effectively. *sed* is quite an advanced filter and all its options can be seen on its man page.

Syntax:

```
sed  [path]
```

The expression we have used above is very basic and is of the form **'s/search/replace/g'**

```
$sed 's/unix/linux/' geekfile.txt
```

10. nl : nl is used to number the lines of our text data.

Syntax:

```
nl [-options] [path]
$ cat list.txt
apples
oranges
potatoes
lemons
garlic
$ nl list.txt
     1 apples
     2 oranges
     3 potatoes
     4 lemons
     5 garlic
```

Shell Scripting

Usually, shells are interactive, which means they accept commands as input from users and execute them. However, sometimes we want to execute a bunch of commands routinely, so we have to type in all commands each time in the terminal.

As a shell can also take commands as input from file, we can write these commands in a file and can execute them in shell to avoid this repetitive work. These files are called **Shell Scripts** or **Shell Programs**. Shell scripts are similar to the batch file in MS-DOS. Each shell script is saved with **`.sh`** file extension e.g., **myscript.sh.**

A shell script has syntax just like any other programming language. If you have any prior experience with any programming language like Python, C/C++ etc. It would be very easy to get started with it.

A shell script comprises the following elements –

- Shell Keywords – if, else, break etc.
- Shell commands – cd, ls, echo, pwd, touch etc.
- Functions
- Control flow – if..then..else, case and shell loops etc.

Why do we need shell scripts?

There are many reasons to write shell scripts:

- To avoid repetitive work and automation
- System admins use shell scripting for routine backups.

- System monitoring
- Adding new functionality to the shell etc.

Some Advantages of shell scripts

- The command and syntax are exactly the same as those directly entered in the command line, so programmers do not need to switch to entirely different syntax
- Writing shell scripts are much quicker
- Quick start
- Interactive debugging etc.

Some Disadvantages of shell scripts

- Prone to costly errors, a single mistake can change the command which might be harmful.
- Slow execution speed
- Design flaws within the language syntax or implementation
- Not well suited for large and complex task
- Provide minimal data structure unlike other scripting languages. etc.

Simple demo of shell scripting using Bash Shell

If you work on a terminal, something you traverse deep down in directories. Then for coming few directories up in path we have to execute a command like this as shown below to get to the "python" directory:

get to the "python" directory:

It is quite frustrating, so why not we can have a utility where we just have to type the name of directory and we can directly jump to that without executing the "**cd ../**" command again and again. Save the script as "**jump.sh**"

```
# !/bin/bash

# A simple bash script to move up to desired directory level directly

function jump()
{
    # original value of Internal Field Separator
    OLDIFS=$IFS

    # setting field separator to "/"
    IFS=/

    # converting working path into array of directories in path
    # eg. /my/path/is/like/this
    # into [, my, path, is, like, this]
```

```
    path_arr=($PWD)

    # setting IFS to original value
    IFS=$OLDIFS

    local pos=-1

    # ${path_arr[@]} gives all the values in path_arr
    for dir in "${path_arr[@]}"
    do
        # find the number of directories to move up to
        # reach at target directory
        pos=$[$pos+1]
        if [ "$1" = "$dir" ];then

            # length of the path_arr
            dir_in_path=${#path_arr[@]}

            #current working directory
            cwd=$PWD
            limit=$[$dir_in_path-$pos-1]
            for ((i=0; i<limit; i++))
            do
                cwd=$cwd/..
            done
            cd $cwd
            break
        fi
    done
}
```

For now, we cannot execute our shell script because it does not have permissions. We have to make it executable by typing following command –

$ chmod +x path/to/our/file/jump.sh

Now to make this available on every terminal session, we have to put this in "**.bashrc**" file. "**.bashrc**" is a shell script that Bash shell runs whenever it is started interactively. The purpose of a .bashrc file is to provide a place where you can set up variables, functions, and aliases, define our prompt, and define other settings that we want to use whenever we open a new terminal window.
Now open the terminal and type the following command:

$ echo "source ~/path/to/our/file/jump.sh">> ~/.bashrc

Now open your terminal and try out new "jump" functionality by typing following command-

$ jump dir_name

The Directory Stack

The directory stack is a list of recently-visited directories. The `pushd` builtin adds directories to the stack as it changes the current directory, and the `popd` builtin removes specified directories from the stack and changes the current directory to the directory removed. The `dirs` builtin displays the contents of the directory stack. The current directory is always the "top" of the directory stack.
The contents of the directory stack are also visible as the value of the `DIRSTACK` shell variable.

Directory Stack

The directory stack is a list of directories you have previously navigated to. The contents of the directory stack can be seen using the `dirs` command. Directories are added to the stack when changing to a directory using the `pushd` command and removed with the `popd` command.

The current working directory is always on the top of the directory stack. The current working directory is the directory (folder) in which the user is currently working in. Each time you interact with the command line, you are working within a directory.

The `pwd` command allows you to find out what directory you are currently in.

When navigating through the file system, use the `Tab` key to autocomplete the names of directories. Adding a slash at the end of the directory name is optional.

`pushd`, `popd` and `dirs` are shell builtins, and its behavior may slightly differ from shell to shell. We will cover the Bash builtin version of the commands.

`pushd` Command

The syntax for the `pushd` command is as follows:

```
pushd [OPTIONS] [DIRECTORY]
```
Copy

For example to save the current directory to the top of the directory stack and change to `/var/www` you would type:

```
~$ pushd /var/www
```

On success, the command above will print the directory stack. `~` is the directory in which we executed the `pushd` command. The tilde symbol `~` means home directory.

`pushd` first saves the current working directory to the top of the stack and then navigates to the given directory. As the current directory must always be on the top of the stack, once changed the new current directory goes to the top of the stack but it is not saved in the stack. To save it you must invoke `pushd` from it. If you use `cd` to change to another directory, the top item of the stack will be lost,

Let's add another directory to the stack:

```
/var/www$ pushd /opt
```
Copy

```
/opt /var/www ~
```
Copy

To suppress changing to directory, use the `-n` option. For example, to add the `/usr/local` directory to the stack but not change into it you would type:

```
/opt$ pushd -n /usr/local
```
Copy

As the current directory (which is always in the top) is not changed, the `/usr/local` directory is added second from the top of the stack:

```
/opt /usr/local /var/www ~
```
Copy

The `pushd` accepts two options, `+N` and `-N` that allows you to navigate to `Nth` directory of the stack. The `+N` option changes to `Nth` element of the stack list counting from left to right starting with zero. When `-N` is used the direction of the count is from right to left.

To better illustrate the options, let's print the current directory stack:

```
/opt$ dirs -l -v
```
Copy

The output will show an indexed list of the directory stack:

```
 0  /opt
 1  /usr/local
 2  /var/www
 3  /home/linuxize
```
Copy

When used without any argument, `pushd` will toggle the top two directories and makes the new top the current directory. This is the same as when using the `cd -` command.

`popd` Command

The `popd` command takes the form:

```
popd [OPTIONS]
Copy
```

When used with no argument, `popd` removes the top directory from the stack and navigates to the new top directory.

Let's say we have the following directory stack:

```
/opt /usr/local /var/www /etc/nginx ~
Copy
```

If you run the `popd` command it will remove the `/opt` from the stack and change to the `/usr/local` directory:

```
/opt$ popd
Copy
```

The output will show the new directory stack:

```
/usr/local /var/www /etc/nginx ~
Copy
```

The `-n` option suppresses the default directory change and removes the second item from the stack:

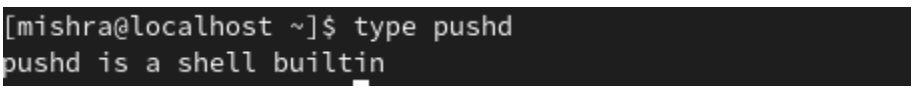
```
/opt$ popd -n
```

pushd Command in Linux

pshd is a shell built-in command which allows us to easily manipulate the directory stack. This appends a directory to the top of the directory stack, or rotates the stack, making the new top of the stack the present working directory. The "d" in pushd stands for the directory as it pushes the directory path onto the stack. The directory stack increases in size after each pushd command. This stack is based on the Last In First Out (LIFO) principle. This command has an exit status 0 i.e, it returns success unless an invalid argument is supplied or the directory change fails.

Syntax of pushd command:

```
pushd [directory]
```



the type command shows that pushd is a shell built-in command.

How pushd command functions?

- When pushd [directory] command is executed, the directory specified becomes the present working directory.
- The path and name of the directory are added to the top of the directory stack.
- The directory stack is displayed as a space-separated list of directories.
- If pushd command is executed without any directory name, then the directory at the top of the stack becomes the current working directory i.e, the first two directories exchange their position in the directory stack.

Advantage of pushd command over cd command

It is a very powerful tool for operation on directories as it stores the directories in the stack. Consider nested directories. We can navigate between the dictionaries using the cd command. But suppose, you are in the fourth directory. Then to navigate to the second directory, the **cd command** has to be used twice. But by using pushd command, it can be achieved in one step. In one step, we can navigate from any directory in the stack to another directory in the stack. Directory manipulation becomes easier and efficient.

Working with pushd command

1. Adding directories using pushd command:

pushd command pushes directories onto a stack. Execute the following commands:

```
pushd ~/Desktop
```

Now, Desktop becomes the present working directory and is pushed onto the stack and the list of directories is displayed. The same operations will be performed for the other commands below.

```
pushd ~/Templates

pushd ~/Videos

pushd ~/Downloads

pushd ~/Music

pushd ~/Downloads
```

Unix / Linux - Processes Management

We will discuss in detail about process management in Unix. When you execute a program on your Unix system, the system creates a special environment for that program. This environment contains everything needed for the system to run the program as if no other program were running on the system.

Whenever you issue a command in Unix, it creates, or starts, a new process. When you tried out the **ls** command to list the directory contents, you started a process. A process, in simple terms, is an instance of a running program.

The operating system tracks processes through a five-digit ID number known as the **pid** or the **process ID**. Each process in the system has a unique **pid**.

Pids eventually repeat because all the possible numbers are used up and the next pid rolls or starts over. At any point of time, no two processes with the same pid exist in the system because it is the pid that Unix uses to track each process.

Starting a Process

When you start a process (run a command), there are two ways you can run it −

- Foreground Processes
- Background Processes

Foreground Processes

By default, every process that you start runs in the foreground. It gets its input from the keyboard and sends its output to the screen.

You can see this happen with the **ls** command. If you wish to list all the files in your current directory, you can use the following command −

```
$ls ch*.doc
```

This would display all the files, the names of which start with **ch** and nd with **.doc** −

```
ch01-1.doc   ch010.doc  ch02.doc    ch03-2.doc
ch04-1.doc   ch040.doc  ch05.doc    ch06-2.doc
ch01-2.doc   ch02-1.doc
```

The process runs in the foreground, the output is directed to my screen, and if the **ls** command wants any input (which it does not), it waits for it from the keyboard.

While a program is running in the foreground and is time-consuming, no other commands can be run (start any other processes) because the prompt would not be available until the program finishes processing and comes out.

Background Processes

A background process runs without being connected to your keyboard. If the background process requires any keyboard input, it waits.

The advantage of running a process in the background is that you can run other commands; you do not have to wait until it completes to start another!

The simplest way to start a background process is to add an ampersand (**&**) at the end of the command.

```
$ls ch*.doc &
```

This displays all those files the names of which start with **ch** and end with **.doc** −

```
ch01-1.doc   ch010.doc  ch02.doc    ch03-2.doc
ch04-1.doc   ch040.doc  ch05.doc    ch06-2.doc
ch01-2.doc   ch02-1.doc
```

Here, if the **ls** command wants any input (which it does not), it goes into a stop state until we move it into the foreground and give it the data from the keyboard.

That first line contains information about the background process - the job number and the process ID. You need to know the job number to manipulate it between the background and the foreground.

Press the Enter key and you will see the following −

```
[1]   +   Done                 ls ch*.doc &
$
```

The first line tells you that the **ls** command background process finishes successfully. The second is a prompt for another command.

Listing Running Processes

It is easy to see your own processes by running the **ps** (process status) command as follows −

```
$ps
PID     TTY     TIME      CMD
18358    ttyp3   00:00:00   sh
18361    ttyp3   00:01:31   abiword
18789    ttyp3   00:00:00   ps
```

Stopping Processes

Ending a process can be done in several different ways. Often, from a console-based command, sending a CTRL + C keystroke (the default interrupt character) will exit the command. This works when the process is running in the foreground mode.

If a process is running in the background, you should get its Job ID using the **ps** command. After that, you can use the **kill** command to kill the process as follows −

```
$ps -f
UID     PID  PPID C STIME    TTY   TIME CMD
```

amrood 6738 3662 0 10:23:03 pts/6 0:00 first_one

amrood 6739 3662 0 10:22:54 pts/6 0:00 second_one

amrood 3662 3657 0 08:10:53 pts/6 0:00 -ksh

amrood 6892 3662 4 10:51:50 pts/6 0:00 ps -f

$kill 6738

Terminated

Here, the **kill** command terminates the **first_one** process. If a process ignores a regular kill command, you can use **kill -9** followed by the –

$kill -9 6738

Terminated

Parent and Child Processes

Each unix process has two ID numbers assigned to it: The Process ID (pid) and the Parent process ID (ppid). Each user process in the system has a parent process.

Initializing a process

A process can be run in two ways:

Method 1: Foreground Process : Every process when started runs in foreground by default, receives input from the keyboard, and sends output to the screen. When issuing pwd command

```
$ ls pwd
```

Output:

```
$ /home/geeksforgeeks/root
```

When a command/process is running in the foreground and is taking a lot of time, no other processes can be run or started because the prompt would not be available until the program finishes processing and comes out.

Method 2: Background Process: It runs in the background without keyboard input and waits till keyboard input is required. Thus, other processes can be done in parallel with the process running in the background since they do not have to wait for the previous process to be completed.

Adding & along with the command starts it as a background process

```
 $ pwd &
```

Since pwd does not want any input from the keyboard, it goes to the stop state until moved to the foreground and given any data input. Thus, on pressing

Enter:

Output:

```
[1]   +   Done                    pwd

$
```

That first line contains information about the background process – the job number and the process ID. It tells you that the ls command background process finishes successfully. The second is a prompt for another command.

Tracking ongoing processes

ps (Process status) can be used to see/list all the running processes.

```
$ ps

PID        TTY       TIME         CMD
19         pts/1     00:00:00     sh
24         pts/1     00:00:00     ps
```

Types of Processes

1. **Parent and Child process :** The 2nd and 3rd column of the ps –f command shows process id and parent's process id number. For each user process, there's a parent process in the system, with most of the commands having shell as their parent.
2. **Zombie and Orphan process :** After completing its execution a child process is terminated or killed and SIGCHLD updates the parent process about the termination and thus can continue the task assigned to it. But at times when the parent process is killed before the termination of the child process, the child processes become orphan processes, with the parent of all processes "init" process, becomes their new pid.
   A process which is killed but still shows its entry in the process status or the process table is called a zombie process, they are dead and are not used.
3. **Daemon process :** They are system-related background processes that often run with the permissions of root and services requests from other processes, they most of the time run in the background and wait for

S